

Silo-Slayer Syndicate (SSS): App Summary

Repo source: /Users/rk/gh/andykerber/syndicate

What It Is

Silo-Slayer Syndicate is a hybrid Python and JavaScript AI-agent framework focused on "information liberation" across disconnected tools. It combines session-persistent agents, human disambiguation workflows, and MCP-based tool integrations so natural language can drive routing and automation.

Who It's For

Primary persona: a power user/knowledge worker managing notes, tasks, and financial workflows across many apps and repos (explicitly exemplified in repo context by Randy Kerber).

What It Does

- Runs session-persistent AI agents with conversation memory via SQLite sessions.
- Uses natural language parameter extraction and "English as programming language" routing patterns.
- Supports human-in-the-loop disambiguation/approval using async request-response queues.
- Exposes human interaction as MCP tools via a FastMCP server (`human_input_server.py`).
- Integrates tool access across custom JS tools, MCP servers, and OpenAI built-ins through a central registry.
- Includes a Hedgeye data pipeline (RR and CR) that parses emails, merges/enriches data, and generates CSV/plots.

How It Works (Architecture)

- Agent layer (Python): `SSSAgent` wraps OpenAI Agents SDK (`Agent`, `Runner`, `SQLiteSession`) for persistent multi-turn chat and tool use.
- Human interaction service: `HumanQueue` writes request files, waits for responses, archives interactions, and can send push notifications.
- MCP service layer: Python MCP servers (e.g., `human_input_server.py`) expose disambiguation/approval tools over stdio transport.
- Tool integration layer (JS): `ToolRegistry` discovers MCP tools from configured servers and unifies custom + MCP + OpenAI tool definitions.
- Data flow: user input -> agent intent/parameter extraction -> tool call or human clarification -> routed action/output (apps/files) or pipeline artifacts.

How To Run (Minimal Getting Started)

- From repo root, set env file: `cp config/shared/.env.example config/shared/.env` (fill required keys).
- Python setup/tests: `cd python && uv sync && uv run pytest tests/`.
- JavaScript setup/build: `cd js && npm install && npm run build`.
- Hedgeye quick run (documented pipeline entrypoint): `cd python && uv run python scripts/hedgeye/run_both_pipelines.py`.
- Single unified "run the whole app" command: Not found in repo.